Black-Box Configuration Search for Generalized Disjunctive Programming

Albert Joon Lee [1], Woosik Kim [2], and David E. Bernal Neira [1,*]

[1]Davidson School of Chemical Engineering, Purdue University, West Lafayette, IN 47907, USA

[2]Elmore Family School of Electrical and Computer Engineering, Purdue University, West Lafayette, IN 47907, USA

*Email: dbernaln@purdue.edu

Abstract

Selecting configurations in generalized disjunctive programming (GDP) can require repeated, costly subproblem evaluations. Existing external integer representations allow these logical choices to be explored by derivative-free search. We ask whether RBFOpt, a surrogate-based optimizer, improves target-hit rates over budget-limited discrete steepest descent under common initial configurations and evaluation budgets. Tests cover 21 Grid problems and four process GDP families, with Branch-and-Model (BAM), factorization-machine-based optimization with simulated annealing (FMQA-SA), and sampling controls as additional comparisons. Here we show that RBFOpt can improve target-hit rates under limited budgets, although its computational cost can be higher. On the continuous stirred-tank reactor (CSTR) networks, RBFOpt reaches the target in 45 of 60 runs, compared with 14 for discrete steepest descent. On Grid at an evaluation budget of $B = 100$, RBFOpt reaches the target in 81.4% of runs, compared with 48.1% for discrete steepest descent, 70.0% for BAM, and 21.4% for uniform sampling. For the dynamic process benchmark, RBFOpt reaches the target in all ten runs at the smallest tested budget, $B = 12$, whereas discrete steepest descent first does so at $B = 30$. Completion costs are estimated from recorded searches using stored objective responses and separately measured subproblem costs. These results support RBFOpt as an alternative for GDP configuration search under limited evaluation budgets. Its benefit depends on the problem, failure responses, and search overhead.

Keywords: Generalized disjunctive programming; Black-box optimization; Derivative-free optimization; Surrogate modeling; Process systems engineering; Computational benchmarking.

1 Introduction

In chemical process design, evaluating a single candidate configuration can require costly optimization or simulation, making the choice of which configuration to explore next a central computational challenge. Process design involves discrete decisions, such as equipment selection, the number of processing stages, and operating modes, along with continuous variables such as flow rates, temperatures, and equipment sizes. Generalized disjunctive programming (GDP) provides a framework to represent these decisions through Boolean variables, logical propositions, and conditional constraints (Raman and Grossmann, 1994). In logic-based GDP solution methods, fixing the Boolean variables associated with a selected configuration determines which constraints remain active and yields a reduced optimization subproblem. When all discrete decisions are fixed, this subproblem can be formulated as a nonlinear programming (NLP) problem. Exploring multiple configurations may therefore require repeatedly solving NLP subproblems, which can be computationally expensive. Consequently, identifying high-quality configurations with fewer evaluations of NLP subproblems is an important objective in solving GDP problems.

Conventional GDP solution methods generally rely on mixed-integer nonlinear programming (MINLP) reformulations or logic-based decomposition. The Big-M and Hull reformulations transform disjunctive constraints into algebraic formulations (Grossmann and Lee, 2003; Raman and Grossmann, 1994), while logic-based approaches select discrete configurations and solve the corresponding reduced subproblems (Bergamini et al., 2005; Türkay and Grossmann, 1996). In logic-based approaches, the strategy used to select discrete configurations can significantly affect the number of subproblem evaluations required.

Discrete-Steepest Descent (D-SDA) was previously applied to process superstructure optimization by Liñán et al. (2020). Building on this approach, our previous work introduced the Logic-Based Discrete-Steepest Descent (LD-SDA), which represents ordered Boolean decisions using external integer variables (Ovalle et al., 2025). LD-SDA explores the resulting integer space through neighborhood search and discrete line search, evaluating selected configurations by solving their corresponding reduced optimization subproblems. However, as a local search method, LD-SDA may converge to different locally optimal solutions depending on the starting point and the selected neighborhood (Peng et al., 2024).

Surrogate-based derivative-free optimization (DFO) uses accumulated objective observations to guide new evaluations (Larson et al., 2019). RBFOpt builds radial basis function surrogates and balances predicted objective values with exploration (Costa and Nannicini, 2018). We apply it to the existing GDP external integer representation. We ask whether RBFOpt reaches reference targets more often than budget-limited D-SDA under common initial configurations and limited evaluation budgets. The comparison also includes Branch-and-Model (BAM) (Ma et al., 2023) and a variant of the factorization machine with quantum annealing (FMQA) framework (Kitai et al., 2020). We use classical simulated annealing instead of quantum annealing in this variant and denote it FMQA-SA. Uniform sampling provides a baseline without objective-dependent candidate selection.

Fewer evaluations do not necessarily imply lower computational cost, because evaluation costs vary and candidate selection incurs overhead (Moré and Wild, 2009). We therefore compare how often each method reaches the target, along with configuration-evaluation costs and search overhead. The methods share precomputed configuration responses, while response handling and stopping remain implementation-specific. The contributions are as follows:

- An evaluation of RBFOpt on the existing GDP external integer representation using 21 Grid-GDP benchmarks and four published process GDP families, with common initial configurations, evaluation budgets, and reference targets.
- An analysis of where RBFOpt improves target attainment and how configuration-evaluation costs and search overhead affect its value relative to the tested alternatives.

2 Background

2.1 Generalized Disjunctive Programming and Fixed-Configuration Evaluation

GDP represents operational and design alternatives using Boolean variables and associated algebraic constraints (Raman and Grossmann, 1994). The original continuous variables have domain $\mathcal{X} = [\underline{\mathbf{x}}, \overline{\mathbf{x}}] \subseteq \mathbb{R}^{n_x}$. The original integer variables have domain $\mathcal{Z} = [\underline{\mathbf{z}}, \overline{\mathbf{z}}] \cap \mathbb{Z}^{n_z}$. Let K index the disjunctions and D_k index the disjuncts in disjunction $k \in K$. Each disjunct $i \in D_k$ has a Boolean variable Y_{ik}. The GDP formulation is given by

$$
\begin{aligned}
\min_{\mathbf{x},\mathbf{z},\mathbf{Y}} \quad & f(\mathbf{x},\mathbf{z}) \\
\text{s.t.} \quad & \mathbf{g}(\mathbf{x},\mathbf{z}) \le \mathbf{0}, \\
& \bigvee_{i \in D_k} \begin{bmatrix} Y_{ik} \\ \mathbf{h}_{ik}(\mathbf{x},\mathbf{z}) \le \mathbf{0} \end{bmatrix}, \qquad k \in K, \\
& \Omega(\mathbf{Y}) = \text{True}, \\
& \mathbf{x} \in \mathcal{X}, \quad \mathbf{z} \in \mathcal{Z}, \quad \mathbf{Y} \in \{\text{False}, \text{True}\}^{n_y}.
\end{aligned}
\tag{GDP}
$$

Here, f is the objective function, $\mathbf{g}$ represents the global constraints, and $\mathbf{h}_{ik}$ represents the constraints associated with each disjunct. The operator $\veebar$ requires that exactly one disjunct be selected within each disjunction. Logical propositions $\Omega(\mathbf{Y})$ impose additional relationships between Boolean variables.

Following the external-variable reformulation of Ovalle et al. (2025), each ordered group of m_j Boolean alternatives is represented by an external integer variable $\xi_j \in \{1, \ldots, m_j\}$, which identifies the selected alternative. Let n_E denote the number of external integer variables, and $\mathcal{Y}_{\text{logic}}$ the set of Boolean assignments that satisfy the conditions `exactly-one`, and $\Omega(\mathbf{Y})$. The external-variable lattice and its logically admissible subset are

$$\begin{aligned} \mathcal{L}_E &= \prod_{j=1}^{n_E} \{1, \ldots, m_j\}, \\ \mathcal{E}_{\text{adm}} &= \{\boldsymbol{\xi} \in \mathcal{L}_E : \mathbf{Y}(\boldsymbol{\xi}) \in \mathcal{Y}_{\text{logic}}\}. \end{aligned} \tag{1}$$

For each logically admissible configuration $\boldsymbol{\xi}$, the corresponding Boolean variables are fixed, yielding the following reduced optimization subproblem:

$$\begin{aligned} \Phi(\boldsymbol{\xi}) = \inf_{\mathbf{x},\mathbf{z}} \quad & f(\mathbf{x}, \mathbf{z}) \\ \text{s.t.} \quad & \mathbf{g}(\mathbf{x}, \mathbf{z}) \leq \mathbf{0}, \\ & \mathbf{h}_{ik}(\mathbf{x}, \mathbf{z}) \leq \mathbf{0} \quad \text{for all } (i,k) \text{ with } Y_{ik}(\boldsymbol{\xi}) = \text{True}, \\ & \mathbf{x} \in \mathcal{X}, \quad \mathbf{z} \in \mathcal{Z}. \end{aligned} \tag{Sub}$$

Here, $\Phi(\boldsymbol{\xi})$ represents the optimal objective value of the reduced subproblem. The reduced subproblem is an NLP when all remaining decision variables are continuous. If the reduced subproblem is infeasible, we define $\Phi(\boldsymbol{\xi}) = +\infty$.

When the external-variable mapping represents all logically admissible Boolean configurations, the original GDP can be expressed as the following upper-level optimization problem (Ovalle et al., 2025):

$$\min_{\boldsymbol{\xi} \in \mathcal{E}_{\text{adm}}} \Phi(\boldsymbol{\xi}). \tag{2}$$

A logically admissible configuration may still yield an infeasible reduced subproblem.

2.2 Conventional GDP Solution Approaches

Big-M and Hull reformulations transform GDP models into mixed-integer nonlinear programming (MINLP) formulations by associating Boolean variables Y_{ik} with binary variables y_{ik} and translating logical propositions into algebraic constraints (Grossmann and Lee, 2003; Raman and Grossmann, 1994). In the Big-M reformulation, the disjunctive constraints are expressed as $\mathbf{h}_{ik}(\mathbf{x}, \mathbf{z}) \leq \mathbf{M}_{ik}(1 - y_{ik})$, where the inequality is applied componentwise. The Big-M values must be sufficiently large to render the corresponding constraints redundant when $y_{ik} = 0$, but excessively large values can weaken the continuous relaxation and cause numerical difficulties.

Hull reformulations introduce disaggregated variables, linking constraints, and bounds scaled by binary indicators (Grossmann and Lee, 2003). Disjunctive constraints are imposed on the disaggregated variables when $y_{ik} = 1$, while $y_{ik} = 0$ forces these variables to zero, making the corresponding disjunct inactive. For convex disjunctive sets, this formulation can provide a stronger continuous relaxation than Big-M, although it introduces additional variables and constraints. For nonconvex GDP problems, the Hull reformulation does not guarantee a convex reformulated model or global optimality (Chen et al., 2022).

Logic-based methods address GDP problems directly by exploiting their logical and disjunctive structure to select discrete configurations and construct the corresponding optimization subproblems. Logic-based outer approximation (LOA) alternates between a mixed-integer master problem and fixed-configuration NLP subproblems (Türkay and Grossmann, 1996). Under suitable convexity and regularity assumptions, the master problem uses linearization cuts to obtain lower bounds, while feasible solutions to the NLP subproblem provide upper bounds. For nonconvex GDP problems, these linearizations may not provide valid global lower bounds, motivating global logic-based outer approximation (GLOA), which requires globally valid relaxations and subproblem bounds. Bergamini et al. (2005) developed a GLOA approach for process networks using piecewise under- and overestimators.

GDPopt provides implementations of logic-based methods within Pyomo.GDP (Chen et al., 2022; Pyomo Development Team, n.d.). Its relaxation with integer cuts (RIC) method uses a mixed-integer linear programming (MILP) relaxation to select configurations and adds no-good cuts to exclude evaluated discrete assignments, without generating the successive outer-approximation cuts used by LOA (Pyomo Development Team, n.d.).

LD-SDA is an existing alternative that searches ordered external configurations by improving neighborhood moves and discrete line searches (Ovalle et al., 2025). It selects the best improving neighbor, continues along the resulting direction while improvement persists, and resumes neighborhood exploration. The present work uses this discrete-search structure as a reference and compares other configuration-selection strategies and their computational costs.

2.3 Derivative-Free Optimization Solvers

Derivative-free optimization (DFO) uses function observations without requiring derivatives of the objective with respect to the search variables (Larson et al., 2019). For GDP configuration search, the outer method can be derivative-free, even if its evaluation routine uses a derivative-based nonlinear solver. The upper-level search operates over external integer variables that represent the logical decisions in the GDP. Some logical restrictions are embedded in the external-variable reformulation, while additional constraints can be imposed on the configuration space. One approach to DFO uses surrogate models to guide the search. Surrogate methods seek to reduce expensive evaluations by fitting models to observed data, with computational cost arising from both objective evaluations and model-based candidate selection.

2.3.1 RBFOpt

RBFOpt is an open-source surrogate optimizer for bounded black-box problems, with support for continuous, integer, and categorical variables (Costa and Nannicini, 2018; Nannicini, 2021). Given N observed objective values in external integer configurations $\boldsymbol{\xi}^{(1)}, \ldots, \boldsymbol{\xi}^{(N)}$, RBFOpt constructs a radial basis function (RBF) surrogate model of the form

$$\widehat{\Phi}_N(\boldsymbol{\xi}) = \sum_{r=1}^{N} \lambda_r \varphi\Big(\|\boldsymbol{\xi} - \boldsymbol{\xi}^{(r)}\|_2\Big) + p(\boldsymbol{\xi}), \tag{3}$$

where $\widehat{\Phi}_N(\boldsymbol{\xi})$ approximates the objective value for a candidate configuration $\boldsymbol{\xi}$, λ_r are interpolation coefficients determined from the observed values, φ is the RBF kernel evaluated at the Euclidean distance between configurations, and $p(\boldsymbol{\xi})$ is a polynomial tail depending on the selected kernel.

Candidate selection balances low predicted values against exploration away from evaluated points. The metric stochastic response surface method (MSRSM) combines scaled surrogate predictions and distances to existing samples, changing their relative weights during the search (Regis and Shoemaker, 2007). New objective evaluations update the model.

2.3.2 Branch-and-Model (BAM)

Branch-and-Model (BAM) combines domain partitioning with local surrogate construction (Ma et al., 2023). The evaluated points define boxes, and a potentially optimal selection procedure identifies regions for further modeling and search. Within selected boxes, Automated Learning of Algebraic Models (ALAMO) constructs sparse algebraic models from nearby observations (Cozad et al., 2014). BARON optimizes these local models to propose candidates, while box-size exploration maintains broader coverage (Ma et al., 2023; Sahinidis, 1996; The Optimization Firm, 2026). Thus, BARON acts on learned surrogates within BAM; it is not solving the original GDP through this interface. Its support for integer restrictions allows for the configuration domains considered here (The Optimization Firm, 2026).

2.4 Factorization-Machine-Based Optimization

A second-order factorization machine (FM) models individual feature effects and pairwise interactions through shared latent parameters (Rendle, 2010). For a binary encoding $\mathbf{b} \in \{0,1\}^m$ of an external in-

teger configuration $\boldsymbol{\xi}$, the FM fit surrogate is

$$\widehat{\Phi}_{\mathrm{FM}}(\mathbf{b}) = \beta_0 + \sum_{i=1}^{m} \beta_i b_i + \sum_{i=1}^{m-1} \sum_{j=i+1}^{m} \langle \mathbf{a}_i, \mathbf{a}_j \rangle b_i b_j, \tag{4}$$

where $\widehat{\Phi}_{\mathrm{FM}}(\mathbf{b})$ denotes the predicted training response, m is the number of binary features, β_0 is the intercept, β_i is the linear coefficient of b_i, and $\mathbf{a}_i \in \mathbb{R}^r$ is its latent vector of dimension r. The inner product $\langle \mathbf{a}_i, \mathbf{a}_j \rangle$ determines the pairwise interaction coefficient. The model parameters are learned from the evaluated configurations and their corresponding training responses.

For binary inputs, $b_i^2 = b_i$, the fitted model is a binary quadratic model. Excluding the constant term β_0 gives the following quadratic unconstrained binary optimization (QUBO) problem:

$$\min_{\mathbf{b} \in \{0,1\}^m} \mathbf{b}^{\mathsf{T}} \mathbf{Q} \mathbf{b}, \qquad Q_{ij} = \begin{cases} \beta_i, & i = j, \\ \langle \mathbf{a}_i, \mathbf{a}_j \rangle, & i < j, \\ 0, & i > j. \end{cases} \tag{5}$$

The QUBO describes the learned surrogate, not an exact algebraic reformulation of the GDP. Moreover, a binary string need not decode to a logically admissible configuration in the domain.

The factorization machine with quantum annealing (FMQA) framework combines FM surrogate fitting with annealing-based candidate generation (Kitai et al., 2020). Based on observed evaluations, the method fits the FM, samples binary candidates from the resulting quadratic model, and evaluates a selected decoded configuration through the objective-evaluation routine before updating the training data.

The FMQA simulated annealing variant (FMQA-SA) uses classical simulated annealing to sample binary candidates from the learned QUBO. Temperature-dependent acceptance of uphill moves supports exploration, while gradual cooling favors lower-energy solutions (Kirkpatrick et al., 1983). In this study, no quantum hardware was used.

3 Experimental Methods

3.1 Benchmark problems: Grid-GDP

We consider 21 two-dimensional test functions adapted from the Quantum Hamiltonian Descent benchmark collection (Wu Group, n.d.). Each is sampled at 31 equally spaced values per coordinate, giving 961 configurations. For each benchmark function, let $[\ell_x, u_x]$ and $[\ell_y, u_y]$ denote the coordinate bounds, and let $I = J = \{0, \ldots, 30\}$ be the corresponding index sets. The Grid optimizers receive $(i, j) \in \{0, \ldots, 30\}^2$ directly. Relative to the one-based convention in Section 2.1, $(\xi_1, \xi_2) = (i + 1, j + 1)$ is only a relabeling of alternatives. `FMQA-SA` encodes each Grid index in five unsigned bits and rejects decoded values of 31. The physical coordinates are defined as

$$a_i = \ell_x + \frac{i(u_x - \ell_x)}{30}, \qquad b_j = \ell_y + \frac{j(u_y - \ell_y)}{30}. \tag{6}$$

Let f denote the original analytical objective function and let L and U be its minimum and maximum over the 31×31 lattice. Both constants are obtained by exhaustive preprocessing of all 961 lattice points prior to any search. A normalized target tolerance of 10^{-5} corresponds to $10^{-5}(U - L)$ in raw objective units, so equal normalized tolerances do not need to define equally large target sets. Supplementary Table S1 gives target-set sizes for all functions. Full-precision normalization constants are available in the repository

(Supplementary Section S6). Each benchmark function is represented by the following GDP:

$$\begin{aligned}
\min_{x,y,Y^x,Y^y} \quad & F(x,y) = \frac{f(x,y) - L}{U - L} \\
\text{s.t.} \quad & \bigvee_{i \in I} \begin{bmatrix} Y_i^x \\ x = a_i \end{bmatrix}, \qquad \bigvee_{j \in J} \begin{bmatrix} Y_j^y \\ y = b_j \end{bmatrix}, \\
& \text{ExactlyOne}(Y_0^x, \ldots, Y_{30}^x), \\
& \text{ExactlyOne}(Y_0^y, \ldots, Y_{30}^y), \\
& \ell_x \le x \le u_x, \qquad \ell_y \le y \le u_y, \\
& Y_i^x, Y_j^y \in \{\text{False}, \text{True}\}.
\end{aligned} \tag{7}$$

The two exclusive disjunctions contain 62 coordinate-selection Boolean indicators. The objective retains the explicit mathematical expression. Consequently, the algebraic solvers receive the expression and its disjunctive structure, whereas the DFO methods observe only objective values returned at their selected coordinates.

Fixing both coordinate disjunctions fixes x and y, leaving no remaining original continuous degrees of freedom. Thus, the measured fixed-point `IPOPT` (Wächter and Biegler, 2006) evaluations characterize fresh model preparation, solver invocation, and verification, rather than a difficult continuous optimization subproblem. The unfixed objective functions remain nonlinear and generally nonconvex, so Big-M and Hull produce MINLP representations. The complete mathematical definitions and coordinate domains of the 21 benchmark functions are provided in Supplementary Sections S1-S2 (Table S1) (SECQUOIA, 2026; Wu Group, n.d.).

3.2 Process GDP benchmark problems

The process benchmarks include a CSTR network (Liñán et al., 2020), a 17-stage distillation column (Ghouse et al., 2018), and a small batch plant (Kocis and Grossmann, 1988). We use their GDP formulations reported by Ovalle et al. (2025), and the nine-stage nonlinear mode-switching model of Peng et al. (2024). Their objectives minimize the total reactor volume (`CSTR`), condenser and reboiler duties, plus the active tray cost (`Column`), the investment cost (`Small Batch`), and the negative integral of the squared state (`Dynamic`). The external variables represent the reactor/feed cutoff and recycle location (`CSTR`), reflux and boilup positions (`Column`), parallel unit counts (`Small Batch`), and final stages of modes 1 and 2 (`Dynamic`). For these process benchmarks, fixing the external choices leaves only continuous variables, so the resulting subproblems are NLPs. The `CSTR` superstructure contains N potential reactor positions, where $N \in \{5, 10, 15, 20, 25, 30\}$. We denote each `CSTR` instance by its size parameter N, such as `CSTR30` for $N = 30$. `Dynamic` uses 30 finite elements per stage and three Radau collocation points. The reference configurations in external variable coordinates are (N, N) for `CSTR`, $(13, 4)$ for `Column` (physical trays $(14, 5)$), $(2, 2, 1)$ for `Small Batch`, and $(1, 4)$ for `Dynamic`. The full GDP formulations of the `CSTR`, `Column`, and `Small Batch` benchmarks are provided in Appendices A.1-A.3 of Ovalle et al. (2025), respectively, while the `Dynamic` formulation is described in Section IV.C of Peng et al. (2024). External-variable mappings for the three design cases are provided in Appendices B.1, B.2, and B.4 of Ovalle et al. (2025). The `CSTR` volume is reported in liters and the `Small Batch` investment in the source model's dollars. The `Column` uses the source model's weighted cost proxy based on duties and active trays. The model sizes and unit conventions are given in the Supplementary Section S4 (Table S3). `Dynamic` is an abstract benchmark with no calibrated physical units for state or time. Minimizing its negative squared-state integral favors greater integrated state magnitude and tests mode sequencing coupled to dynamic equations.

Table 1 summarizes the original GDP and DFO candidate domains, the number of configurations with `IPOPT`-accepted objective values, the number of configurations with measured evaluation times, and the evaluation budgets. The process searches use frozen objective tables from fixed-configuration `IPOPT` evaluations; only accepted finite objectives are used as valid lookup objectives. The objective, acceptance status, and point cost are taken from the same recorded evaluation attempt for the same model, CSTR size, and external-variable tuple. Measured coverage includes all configurations in each DFO candidate box, including unsuccessful and structural-rejection evaluations. The evaluation budget B limits the total number of

objective evaluations, including initialization, repeated calls, and unsuccessful evaluations. The full GDP box precedes the logical and algebraic feasibility restrictions. `Column` searches $\{8,\dots,15\}\times\{1,\dots,8\}$ (64 coordinates), the logical domain of its original $\{1,\dots,15\}^2$ box. Other cases use their full boxes. The `Column` bounds encode the reflux and boilup placement restrictions; algebraic constraints are still checked during configuration evaluation. All DFO methods use this box, and the `Column` conclusions apply only to it. For `CSTR`, recycle/feed ordering admits $N(N+1)/2$ logical candidates, but not all yield objective values satisfying the solver acceptance criteria defined in Section 3.4. The `CSTR` comparisons do not isolate scalability: evaluation budgets change with size. `Dynamic` permits $z_{E,1} < z_{E,2}$ or $(z_{E,1}, z_{E,2}) = (9, 9)$. `Small Batch` uses all 27 configurations in $\{1,2,3\}^3$, whose decisions count parallel mixer, reactor, and centrifuge units. Of these configurations, 15 are rejected during feasibility-based bound tightening (FBBT) preprocessing.

Table 1: Process GDP domains, `IPOPT`-accepted lookup coverage, measured point-cost coverage, evaluation budgets, and reference objective values. Model sizes and unit conventions are given in Supplementary Table S3.

(a) External-variable domains

Model	Full GDP box	DFO candidate box	Full GDP box size	DFO box size
`Column`	$\{1,\dots,15\}^2$	$\{8,\dots,15\}\times\{1,\dots,8\}$	225	64
`CSTR` N	$\{1,\dots,N\}^2$	$\{1,\dots,N\}^2$	N^2	N^2
`Small Batch`	$\{1,2,3\}^3$	$\{1,2,3\}^3$	27	27
`Dynamic`	$\{1,\dots,9\}\times\{2,\dots,9\}$	$\{1,\dots,9\}\times\{2,\dots,9\}$	72	72

(b) Lookup coverage and evaluation budgets

Model	Finite lookup	Measured coordinates	Evaluation budget B	f_{ref}
`Column`	29	64	13	19346.1
`CSTR5`	15	25	8	3.06201
`CSTR10`	54	100	15	2.88954
`CSTR15`	120	225	22	2.82939
`CSTR20`	210	400	29	2.79941
`CSTR25`	325	625	36	2.78148
`CSTR30`	464	900	43	2.76957
`Small Batch`	12	27	10	167428
`Dynamic`	37	72	12,15,20, 30,37,50	-212.009
Total	1,266	2,438	–	–

The 2,438 measured keys cover the complete DFO candidate domains. The 700 primary process runs (`D-SDA`, `RBFOpt`, `BAM`, `FMQA-SA`, and uniform sampling; 14 model-budget conditions and ten seeds per condition) visit 2,024 distinct model/size/coordinate keys. This union counts each key once across methods, budgets, and seeds.

3.3 Search protocol and compared algorithms

3.3.1 Shared initialization and evaluation budgets

All configuration-search methods and sampling controls use precomputed configuration responses during the search. For process cases, accepted responses are objective values from `IPOPT` under the initialization and acceptance rules in Section 3.4. They need not equal the theoretical subproblem value $\Phi(\boldsymbol{\xi})$.

Each Grid-GDP benchmark uses ten initial designs and evaluation budgets $B \in \{10, 30, 50, 70, 100\}$. The primary Grid comparison comprises four search implementations and two sampling controls, giving $21 \times 10 \times 5 \times 6 = 6{,}300$ runs. The process benchmarks comprise 14 model-budget conditions, each evaluated with ten paired seeds, giving 140 runs per method and 700 runs across the four search implementations and uniform sampling (Table 1). For each problem-budget-seed combination, three distinct initial configurations are generated using seeded Latin hypercube sampling, mapped to integer coordinates, and shared in the same order across all methods and sampling controls. This common design controls the first three evaluations; subsequent sampling, response handling, and stopping rules follow the individual implementations. All process cases use seeds 0-9, with no replacement of failed starts.

Methods may terminate before exhausting B; only Grid `D-SDA` receives the reference target as a stopping criterion. Detailed settings and worker time limits are provided in Supplementary Section S3. All objective callbacks consume the evaluation budget, including initialization, repeated calls, and unaccepted evaluations. Internal cache accesses without objective callbacks are not counted as evaluations. Evaluation acceptance and target attainment are assessed using the recorded evaluation status and original objective values, rather than method-specific penalties. In all process cases, `D-SDA` and `RBFOpt` assign a penalty of 10^6 to unaccepted evaluations. `BAM` represents these evaluations by `NaN`, indicating that no accepted objective value is available. For surrogate training, `FMQA-SA` scales accepted objective values to $[0, 1]$ and assigns unaccepted evaluations a value of 2.

3.3.2 Budget-limited D-SDA

We use a budget-limited implementation of the previously developed `D-SDA` (Liñán et al., 2020), applying its neighborhood search and discrete line search over the GDP external integer lattice as in LD-SDA (Ovalle et al., 2025). It evaluates the unit L_∞ neighborhood in fixed enumeration order, selects the best improving neighbor, and continues along the resulting direction through a discrete line search. Neighborhood and line-search moves require an absolute objective improvement greater than 10^{-5}. If the evaluation budget B is exhausted, the best solution found is retained.

For the process benchmarks, `D-SDA` starts with the best accepted configuration among the three initial points evaluated. For Grid-GDP benchmarks, initial configurations are ranked by objective value and successive descents are attempted while evaluation budget remains. After each completed descent, the search stops if the best observed objective satisfies the common target criterion in Equation (8); otherwise, it tries the next ranked initial configuration. For all process benchmarks, `D-SDA` searches within the declared integer bounds without precomputed feasibility filtering, using the callback accounting and unaccepted-response policy stated above.

3.3.3 RBFOpt

`RBFOpt` uses MSRSM, automatic RBF selection, and the default genetic candidate-search option of version 4.3.0 (Costa and Nannicini, 2018; Nannicini, 2021). All external variables are integers, and the three shared initial configurations are supplied with the additional package initialization disabled. Search stops at the evaluation budget or the native iteration and stall limits. Other surrogate-minimization operations use standalone `Bonmin` 1.8.9 (Bonami et al., 2008) and `IPOPT` 3.14.19 (Wächter and Biegler, 2006) executables. These calls contribute to search time, but not the objective-evaluation budget.

3.3.4 BAM

`BAM` uses local algebraic surrogate models optimized by `BARON` to generate candidate configurations (Ma et al., 2023; The Optimization Firm, 2026). All external variables are integers. The three shared configurations and their responses are supplied without an additional starting point. The remaining evaluation budget is $B - 3$, with other settings at their defaults. On Grid-GDP, the next ten planned evaluations use a Sobol sequence, which distributes points across the domain, subject to the remaining budget. For the process experiments, callback B returns its value normally, and `BAM` terminates through its native evaluation limit.

3.3.5 FMQA-SA

`FMQA-SA` uses the FM training and QUBO construction code of SECQUOIA (n.d.). At each iteration, a new second-order rank-eight FM is fitted to all evaluated configurations. Finite precomputed responses are min-max scaled to $[0, 1]$ (or zero when all finite values are identical), while non-finite evaluations receive a training target of 2.0.

Integer coordinates are encoded in unsigned binary, and the fitted FM is converted to a QUBO. Binary candidates are generated on CPU using D-Wave's classical simulated annealing sampler (D-Wave Systems, n.d.). The lowest-energy decoded candidate within the coordinate bounds that has not been previously evaluated is selected; if none is available, a candidate is drawn uniformly from the remaining unevaluated configurations. Candidates are not screened for the feasibility of logical or nonlinear constraints before

evaluation. The new observation is appended, and FM fitting and candidate generation repeat until the evaluation budget B or the 1,800 s method time limit is reached.

3.3.6 Sampling controls

Uniform sampling evaluates the same three initial configurations and then draws uniformly without replacement from the remaining declared integer domain. It uses a separate deterministic random stream, no objective-dependent selection, and no logical or NLP-feasibility screening before evaluation. The Grid `BAM`-prefix control repeats `BAM`'s ten predetermined sampling points in the same order after the three shared starts, then samples uniformly without replacement, stopping at the budget. This control tests the observed fixed sampling design and does not reconstruct proprietary `BAM` internals.

3.4 Execution environment and algebraic comparisons

The benchmark performance experiments were conducted on the Dantzig server, equipped with an AMD EPYC 7643 processor (48 cores, 2.3 GHz) and 1 TB RAM. All optimization methods and numerical libraries were configured for single-threaded execution. The software environment included `Pyomo` 6.10.0 (Bynum et al., 2021), `RBFOpt` 4.3.0, `BAM` 5.0.0, `PyTorch` 2.8.0 (Paszke et al., 2019), and `SCIP` 9.2.4 (Bolusani et al., 2024). We used the General Algebraic Modeling System (`GAMS`) 52.2.0 (GAMS Development Corporation, 2025) to call `IPOPT` for configuration evaluations and `SCIP` for algebraic comparisons.

Fixed-disjunction NLP evaluations. We fix the original Boolean GDP decisions and evaluate the resulting subproblems using `IPOPT` 3.14.19 with `MUMPS` 5.8.1 (Amestoy et al., 2001). For the Grid-GDP benchmarks, all 20,181 configurations were evaluated individually, while the process benchmarks used fixed-configuration evaluations of the corresponding process models. `IPOPT` used a tolerance of 10^{-8} and a solver time limit of 60 s per configuration. Process acceptance requires a finite objective, a maximum absolute violation of any constraint or variable bound not greater than 10^{-5}, and the accepted solver and termination statuses specified in the Supplementary Section S3.1. Time-limited candidates remain unaccepted even when feasible. `CSTR` sizes 5-30 use the mass-balance/zero-recycle initialization. `Column` and `Small Batch` use their saved initialization files, and `Dynamic` uses fresh clones of the unsolved discretized template.

Conventional GDP comparisons. Conventional comparisons include `GDPOpt` RIC, GLOA, and LOA, as well as Big-M and Hull reformulations. `GDPOpt` uses `SCIP` for both MILP master problems and nonlinear subproblems (Chen et al., 2022; Pyomo Development Team, n.d.). The Big-M and Hull reformulations are solved directly using `SCIP`. All algebraic methods use the original GDP domains, a time limit of 1,800 s per instance, and a relative optimality gap of 10^{-5}.

3.5 Target attainment and cumulative success profiles

Let f_{best} denote the lowest feasible objective value observed so far within the evaluation budget, and let f_{ref} denote the reference objective value. For process benchmarks, f_{best} is computed from evaluations that meet the solver acceptance criteria in Section 3.4. A run reaches the reference target when

$$|f_{\text{best}} - f_{\text{ref}}| \leq 10^{-5} \max\{1, |f_{\text{ref}}|\}. \tag{8}$$

The target attainment measures agreement with the reference value rather than certifying global optimality. For the process experiments, f_{ref} is the lowest accepted objective value obtained from the precomputed fixed-configuration NLP evaluations. Runs that terminate with an execution failure are counted as unsuccessful, even if the target was observed earlier. For process runs, the reported solution is the best accepted configuration recorded within budget, including the final evaluation. Reported objectives and bounds use six significant figures; all target and bound checks use unrounded values. For the full-process-GDP comparison, a feasible objective at or below $f_{\text{ref}} + 10^{-5} \max\{1, |f_{\text{ref}}|\}$ counts as achievement, so solutions better than the reference are also included.

For each evaluation budget B, let $\mathcal{R}_B$ denote the set of problem-seed instances compared across methods, and let s identify a solution method. For each instance $r \in \mathcal{R}_B$, H_{sBr} equals one if the run reaches the target

and zero otherwise. Let E_{sBr} be the index of the first objective evaluation at which the target is reached, with $E_{sBr} = +\infty$ if the target is not reached. At a threshold of N objective evaluations, where $0 \le N \le B$, the absolute evaluation-count profile is

$$C_{sB}^{\mathrm{eval}}(N) = \frac{1}{|\mathcal{R}_B|} \sum_{r \in \mathcal{R}_B} \mathbf{1}\{H_{sBr} = 1,\ E_{sBr} \le N\}. \tag{9}$$

Here, $\mathbf{1}\{\cdot\}$ equals one when the stated conditions hold and zero otherwise, and $|\mathcal{R}_B|$ is the number of problem-seed instances. Thus, $C_{sB}^{\mathrm{eval}}(N)$ gives the fraction of runs that reach the target within N evaluations. For the Grid-GDP benchmarks, $|\mathcal{R}_B| = 210$ at each budget; for each process benchmark and budget, $|\mathcal{R}_B| = 10$. All runs, including target misses, remain in the denominator.

Let $\widehat{T}_{sBr}$ denote the estimated completion time of the full-run, combining the recorded search time with separately measured configuration-evaluation costs as defined in Section 3.6. For a time threshold $t \ge 0$, measured in seconds, the absolute completion-time profile is

$$C_{sB}^{\mathrm{time}}(t) = \frac{1}{|\mathcal{R}_B|} \sum_{r \in \mathcal{R}_B} \mathbf{1}\{H_{sBr} = 1,\ \widehat{T}_{sBr} \le t\}. \tag{10}$$

This profile gives the fraction of runs that reach the target and have an estimated full-run completion time not greater than t. A successful run is counted at its full completion estimate, not at the time of its first target attainment.

Experiments at different evaluation budgets are analyzed separately; a larger-budget trajectory need not extend a smaller-budget trajectory. Both profiles use absolute evaluation counts or times rather than solver-relative cost ratios (Dolan and Moré, 2002). The evaluation count profile is related to the data profiles (Moré and Wild, 2009), but uses raw rather than dimension-normalized evaluation counts. Algebraic results are reported separately because their iterations and subsolver calls are not DFO objective evaluations, and only one run is reported for each problem-method pair.

3.6 Runtime accounting and uncertainty

For a recorded lookup-based run with n objective evaluations and measured lookup-call times, the estimated full-run completion time is

$$\widehat{T}_{\mathrm{complete}} = T_{\mathrm{recorded}} - \sum_{q=1}^{n} t_q^{\mathrm{lookup}} + \sum_{q=1}^{n} c(\boldsymbol{\xi}^{(q)}) + t_{\mathrm{prep}}. \tag{11}$$

Here, T_{recorded} is the measured runtime of the search based on lookup, t_q^{lookup} is the measured lookup-processing interval of its qth objective call, and $\boldsymbol{\xi}^{(q)}$ is the external integer configuration evaluated in that call. The point cost $c(\boldsymbol{\xi}^{(q)})$ is measured separately for the same benchmark model, problem size and configuration, including configuration-specific preparation, preprocessing, invocation of the solver when required and validation. The term t_{prep} accounts for the preparation of a single model that is not already included in the point costs. Retained search time includes surrogate fitting, candidate generation, auxiliary optimization, and recording performed within the method clock. Throughout this manuscript, full-run completion denotes completion of this recorded search stage with reconstructed evaluation costs, excluding final result extraction and export. Every recorded objective call contributes its point cost, including initialization, repeated visits, and preprocessing rejections; the same stored measurement is used each time a configuration is revisited. The reported completion-cost distributions are conditional on this frozen point-cost inventory. They reflect variation in search paths and recorded search overhead, but do not estimate repeated-measurement uncertainty in configuration-evaluation costs.

Grid-GDP timers. Grid-GDP enumeration and objective normalization are excluded from the timing analysis, and $t_{\mathrm{prep}} = 0$ because configuration-specific preparation is included in point costs. Recorded lookup-call times are subtracted from the method runtime for all compared Grid methods.

Process timers. Structurally rejected configurations incur measured preparation and rejection costs without an NLP solution (Supplementary Section S3.1). For `Dynamic`, the original model-template construction and discretization cost is included once per run through t_{prep}; additional campaign-level preparation is not charged repeatedly. All in-budget evaluations have measured point costs, and estimated full-run completion times are available for all 700 primary process runs. These estimates retain the recorded lookup trajectories rather than measuring online optimizer-NLP executions.

Algebraic GDP completion times are measured separately and are not augmented with the DFO point costs. Grid-GDP algebraic timings include per-run preparation, reformulation, solution, and validation, but exclude shared-template preparation and lattice enumeration. Full-process-GDP timings include model-template preparation.

Uncertainty. Supplementary Section S5 defines pointwise 95% exact binomial intervals for the ten-seed success fractions. These intervals assume independent seed-level trials under the fixed protocol and are not simultaneous intervals. Paired comparisons resample intact functions for Grid and paired seeds within each process condition (Supplementary Section S8).

4 Computational Results and Discussion

4.1 Search Performance on Grid-GDP Benchmarks

`RBFOpt` achieves more Grid targets than budget-limited `D-SDA` at every tested budget (Table 2). At $B = 100$, it reaches 171 of 210 targets (81.4%), compared with 101 (48.1%) for `D-SDA` and 45 (21.4%) for uniform sampling. From $B = 30$ onward, it has the highest aggregate attainment among the compared methods.

At $B = 10$, `BAM` and its prefix control each reach 111 targets, compared to 42 for `RBFOpt`. All 111 `BAM` hits occur before surrogate-guided selection: 11 during shared initialization and 100 at two predetermined Sobol points. At $B = 100$, `BAM` adds 23 hits over the prefix control, indicating a benefit from its subsequent search.

Target-set sizes vary between functions (Supplementary Table S1). Excluding Xin-She Yang 3 and the nearly redundant sampled Bohachevsky 2 case preserves the $B = 100$ ordering (Supplementary Section S8).

Table 2: Grid-GDP reference-target hit percentages from the recorded trajectories. Each cell contains 210 runs (21 functions and ten seeds). The evaluation budget includes the three shared initial evaluations, and repeated objective callbacks are counted toward the evaluation budget. `BAM`'s 111 hits at $B = 10$ all precede surrogate-guided selection (11 shared-initialization hits and 100 at two predetermined Sobol points). Per-function integer hit counts at $B = 100$ are available in the repository (Supplementary Section S6).

Budget B	`D-SDA`	`RBFOpt`	`BAM`	`FMQA-SA`	Uniform	`BAM`-prefix control
10	6.2	20.0	52.9	7.6	6.7	52.9
30	35.2	61.9	54.8	22.4	11.9	54.8
50	41.9	73.3	57.6	36.2	13.8	56.7
70	46.7	76.7	62.9	48.6	17.1	57.1
100	48.1	81.4	70.0	61.4	21.4	59.0

Figure 1 shows the cumulative fraction of runs attaining the reference target as a function of the number of objective evaluations, revealing when successful searches first reach the target. Each panel is a separate budget experiment, so trajectories at a larger evaluation budget are not assumed to extend those at a smaller evaluation budget. `FMQA-SA` reaches 129 of 210 targets at $B = 100$, compared with 45 for uniform sampling.

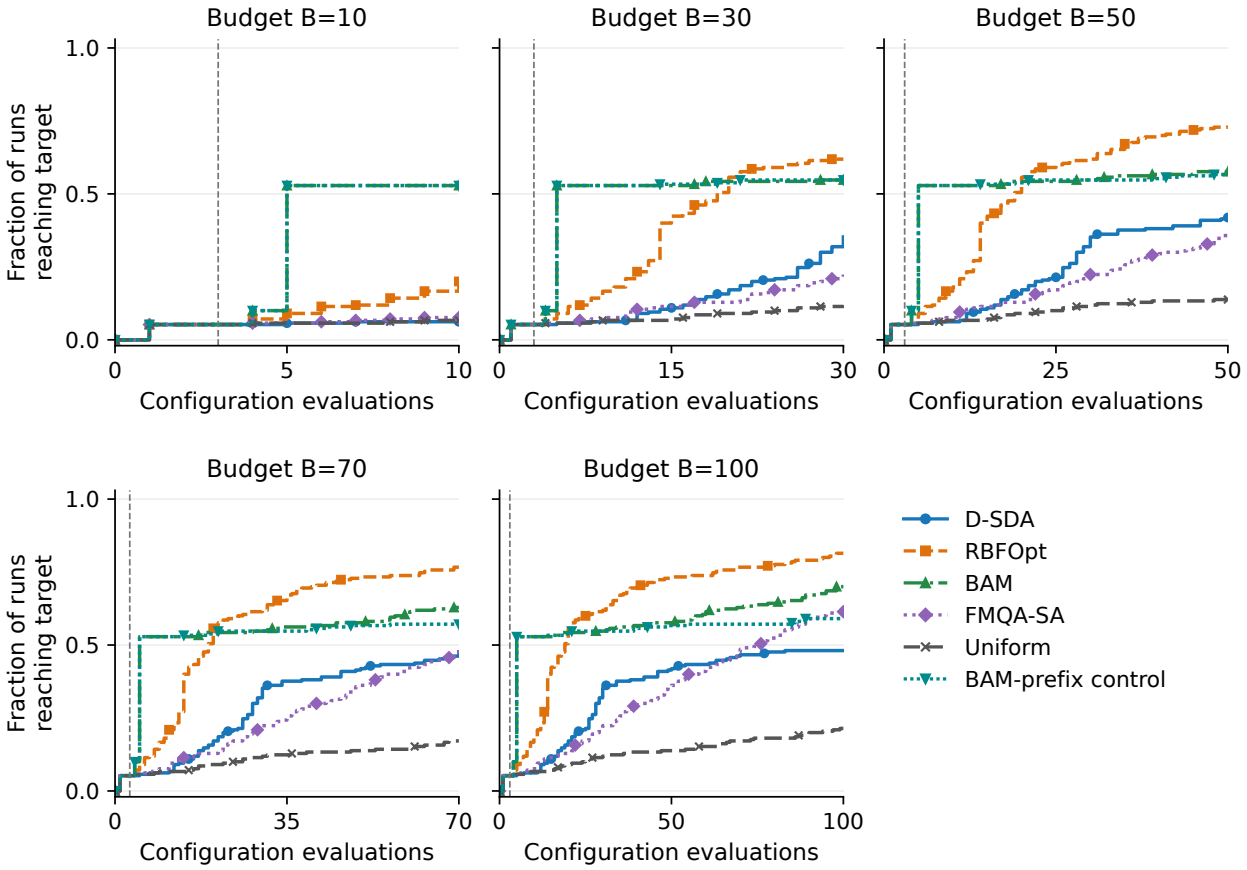


Figure 1: Cumulative success profiles for 21 Grid-GDP functions and ten seeds per function, following Equation (9). The target is the normalized reference value zero within 10^{-5}, as defined by Equation (8). Each panel uses all 210 runs from a separate budget experiment; initial and repeated evaluations count, and target misses remain in the denominator. The dashed line marks the three shared initial evaluations. Uniform sampling and the `BAM`-prefix control are defined in Section 3.3.6. `BAM` rises early, while `RBFOpt` reaches the largest final fraction for $B \geq 30$. D: Why is the x-axis called "Charged evaluations"? What do you mean by sampled controls? Why is the target not specified in the figure? WHat is meant by target hit? Relate them to the equations in the manuscript. The description both has too many words but repeats itself and says nothing. It should follow the structure, describe succintly what is being plotted and then provide a short interpretation. Why is the gray shaded area not starting in zero? You do not define new concepts in the figures (BAM-prefix explained the others not). For which instances is this? Why do we have those extra notes under the legend? A: Clarified the axes and target, linked the profile and controls to their definitions, and removed the shading and legend notes without changing the curves.

4.2 Computational Comparison on Grid-GDP Problems

Higher target attainment comes with additional search cost (Table 3). At $B = 100$, `RBFOpt` has a median estimated full-run completion time of 70.81 s, compared with 24.08 s for `D-SDA` and 64.42 s for `BAM`. `D-SDA` has negligible search overhead and can stop before exhausting the budget, while `RBFOpt` spends a median of 7.58 s on search. For `FMQA-SA`, the median search component is 62.04 s, and the median completion estimate is 124.13 s. Because Grid configurations fix both original coordinates, their measured evaluation costs primarily reflect model preparation, solver invocation, and verification.

Table 3: Grid-GDP results at $B = 100$, with 210 runs per method (21 functions and ten seeds). Target hits reach the normalized reference zero within 10^{-5} (Equation (8)). Times are Q50 (median) [Q25, Q75] in seconds across all runs, including target misses, using linearly interpolated quartiles. Summed configuration-evaluation cost includes the separately measured cost of every evaluation in the full run, including repeated configurations. Retained search time excludes lookup processing; their sum gives estimated full-run completion time (Equation (11)). Component quartiles need not sum to completion quartiles. D: Why only the meadian? Provide the Q25, Q50=median, Q75 intervals. Where did we define what a "point cost" means? Where is the target defined? THe table should be self-defined. A: Added Q25 and Q75 around each median and defined the target and configuration-evaluation cost in the caption, using the same 210 runs per method.

Method	Target hits	Full-run completion (s)	Summed configuration-evaluation cost (s)	Retained search time (s)
		24.08	24.08	0.0039
`D-SDA`	101/210	[15.58, 49.49]	[15.58, 49.48]	[0.0037, 0.0043]
		70.81	60.84	7.58
`RBFOpt`	171/210	[54.83, 93.09]	[44.31, 82.15]	[4.12, 14.82]
		64.42	63.97	0.43
`BAM`	147/210	[48.74, 81.67]	[48.34, 81.22]	[0.37, 0.51]
		124.13	62.34	62.04
`FMQA-SA`	129/210	[105.57, 139.26]	[43.98, 77.52]	[61.58, 62.53]
		60.51	60.51	0.0026
Uniform	45/210	[43.33, 78.60]	[43.33, 78.59]	[0.0026, 0.0027]
		61.81	61.80	0.0026
`BAM`-prefix control	124/210	[40.96, 78.33]	[40.95, 78.33]	[0.0026, 0.0027]

`D-SDA` and `BAM` nevertheless have zero observed hits on four and two functions, respectively; `RBFOpt` and `FMQA-SA` have at least one hit on every function at that evaluation budget. Uniform has seven zero-hit functions, and the prefix control has two.

The algebraic comparisons report one selected result per problem and method; their iterations and subsolver calls are not equivalent to DFO callbacks. The algebraic solver families and configurations are described in Section 3.4. The direct Big-M and Hull reformulations solved with `SCIP` reach all 21 reference targets. The `GDPOpt` runs with `SCIP` reach 16/21, 17/21, and 18/21 targets for RIC, GLOA, and LOA, respectively. These counts combine original non-error runs with reruns of execution errors, so their differences do not establish a method ranking (Supplementary Section S6). All three miss Hölder table, Michalewicz, and Shubert (historical variant). In the diagnosed runs, `GDPOpt` preprocessing rejects independently feasible reference configurations before subproblem evaluation, explaining the target misses despite reported optimal termination. Seven recovered `GDPOpt` runs reach the target before terminating at the time limit. Consequently, the measured completion time of a target-attaining run must be distinguished from a completed global-optimality certification.

4.3 Process GDP Case Studies

4.3.1 Target attainment

Across the six `CSTR` sizes, `RBFOpt` reaches 45 of 60 targets, compared with 14 for `D-SDA` (Table 4). It has more target hits at every tested size under the stated size-dependent budgets. For `Dynamic`, `RBFOpt` reaches all ten targets at every tested budget, starting at $B = 12$, where `D-SDA` reaches four. `D-SDA` reaches all ten from $B = 30$ onward.

The advantage is problem-dependent. On Column at $B = 13$, D-SDA reaches five of ten targets and RBFOpt four within the 64-configuration logical domain. On Small Batch at $B = 10$, D-SDA reaches eight targets, compared with five for RBFOpt.

FMQA-SA reaches 42 of 140 process targets, compared with 37 for uniform sampling. Its aggregate gain over uniform is smaller here than on Grid. Supplementary Section S3.2 reports fallback counts, and Section S8 reports the FMQA diagnostics.

Table 4: Reference-target attainment in the IPOPT-table process experiments. Entries are target hits of the wrapper-reported best-accepted configuration divided by all ten paired seeds at evaluation budget B.

Case	B	D-SDA	RBFOpt	BAM	FMQA-SA	Uniform
Column	13	5/10	4/10	1/10	4/10	1/10
CSTR5	8	2/10	8/10	10/10	3/10	4/10
CSTR10	15	2/10	6/10	1/10	2/10	1/10
CSTR15	22	2/10	7/10	1/10	1/10	2/10
CSTR20	29	3/10	7/10	2/10	1/10	2/10
CSTR25	36	3/10	8/10	2/10	1/10	1/10
CSTR30	43	2/10	9/10	2/10	1/10	1/10
Small Batch	10	8/10	5/10	2/10	6/10	4/10
Dynamic	12	4/10	10/10	0/10	2/10	0/10
Dynamic	15	5/10	10/10	3/10	3/10	3/10
Dynamic	20	8/10	10/10	3/10	3/10	3/10
Dynamic	30	10/10	10/10	4/10	4/10	4/10
Dynamic	37	10/10	10/10	7/10	5/10	5/10
Dynamic	50	10/10	10/10	10/10	6/10	6/10

All 700 primary process runs completed without execution failure. All 70 primary groups use seeds 0-9. The only Column hits for BAM and uniform sampling occur at the shared initial target in seed 6. An accepted target configuration at callback B counts because its coordinates and objective are preserved in the final wrapper summary. BAM also returns that value to its native optimizer; the other methods retain the wrapper stop. The reference is the accepted IPOPT-table minimum, not a global-optimality certificate.

Figure 2 shows the first target-attaining evaluation on successful recorded runs, with misses and execution failures retained in the denominator.

4.3.2 Computational cost

For Dynamic at $B = 50$, search overhead contributes 42.7% of the mean estimated completion time for RBFOpt, compared with less than 0.2% for D-SDA. Dynamic D-SDA executes 221 callbacks across ten $B = 50$ runs, compared with 500 for each of RBFOpt, BAM, and FMQA-SA. Its lower cost reflects shorter paths as well as low search overhead; unsuccessful candidates are evaluated and charged within the common box.

On Column, median completion estimates are 35.44 s for D-SDA and 54.60 s for RBFOpt (Figure 3). For Dynamic at $B = 30$, RBFOpt reaches all ten targets in fewer evaluations than D-SDA (Figure 2). Its median estimated full-run completion time is nevertheless higher: 34.19 s versus 26.62 s (Figure 3). Earlier target discovery therefore need not reduce full-run cost, which includes subsequent evaluations and search overhead. At the smaller budget $B = 12$, RBFOpt reaches all ten targets with a median completion estimate of 12.29 s.

Supplementary Table S4 reports median estimated full-run completion times for all 14 conditions. For CSTR30, RBFOpt reaches nine targets with a median completion estimate of 101.96 s. D-SDA reaches two targets at 53.12 s, and BAM reaches two at 115.41 s.

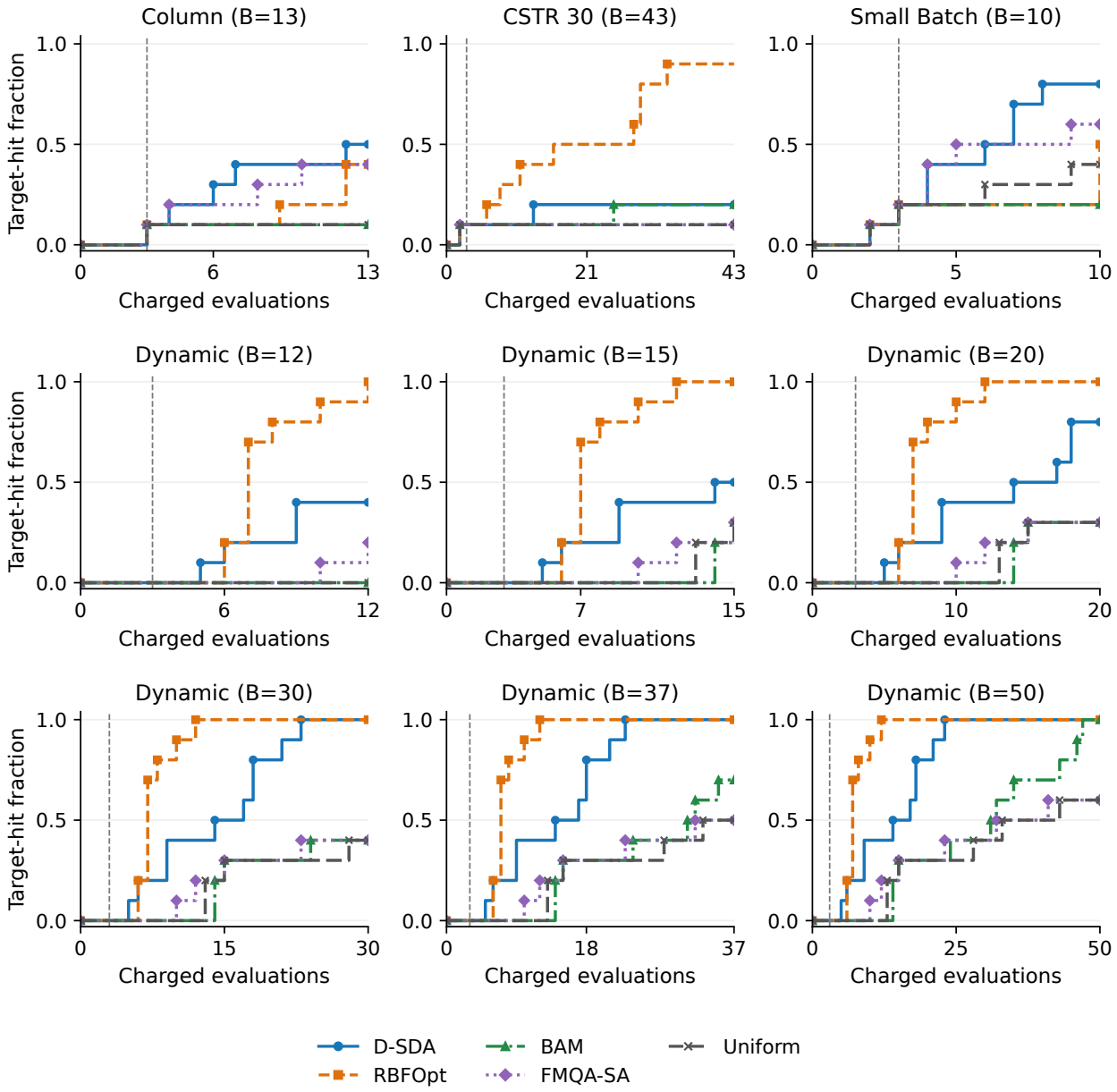


Figure 2: Cumulative target attainment versus charged evaluations for nine illustrative process conditions, using the wrapper-reported success definition in Section 3.5. Every method-condition group retains all ten seeds, including target misses. Initialization, unsuccessful evaluations, and repeated callbacks consume the budget. Steps mark the first accepted target observation on successful runs. The gray vertical line at three evaluations marks the end of the shared initialization. Line styles and markers distinguish methods. All 14 conditions appear in Table 4.

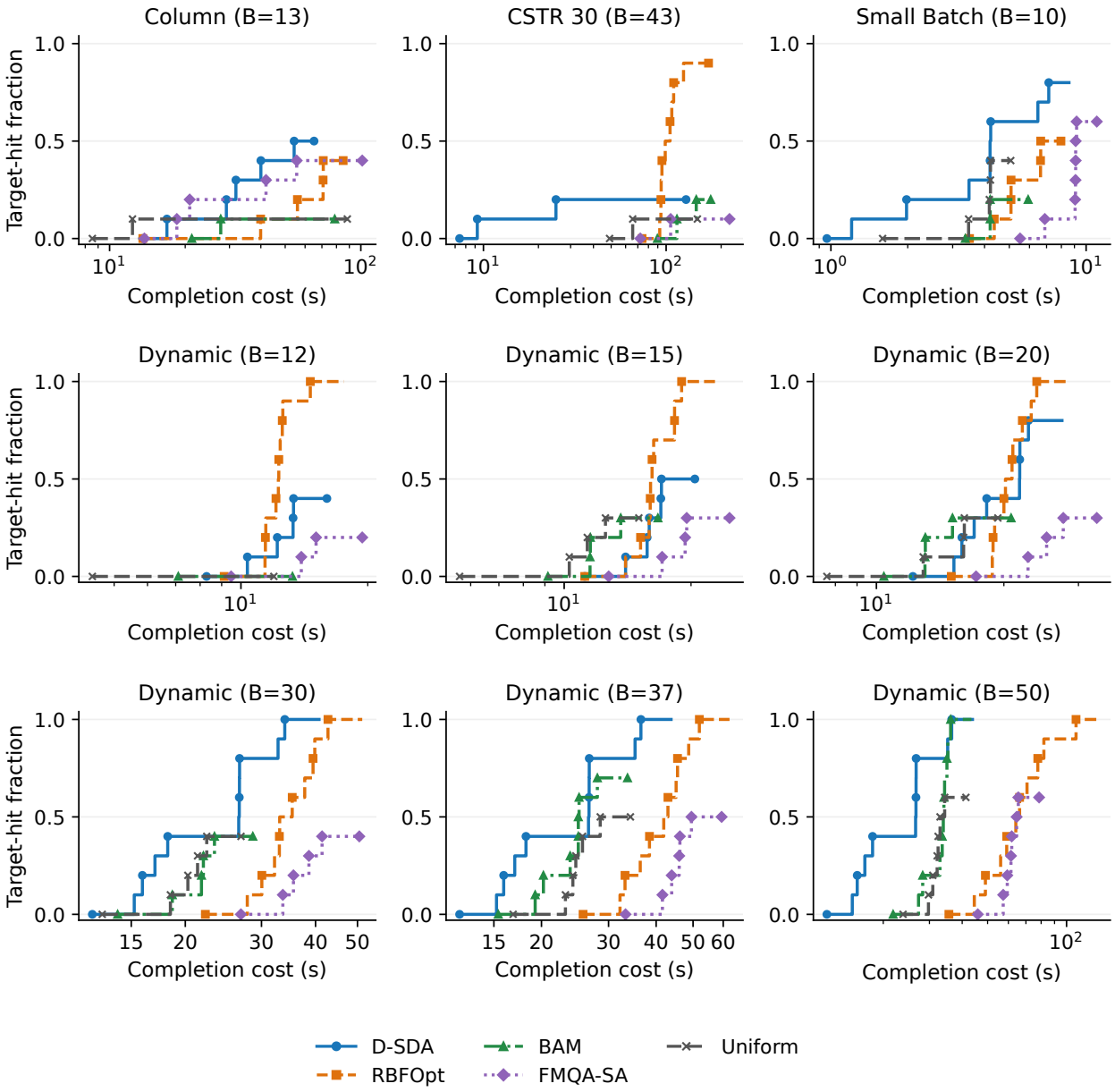


Figure 3: Cumulative success profiles versus estimated full-run completion time for nine process conditions, defined by Equation (10). Each curve includes all ten seeds in its denominator and counts successful runs at their full-run completion estimates. Target attainment follows Equation (8); completion times follow Equation (11) and are plotted on a logarithmic scale. D: We cannot have the definition of a figure be defined in the supplementary material. Either include it in the main body or remove the figure A: Addressed by linking the caption to the profile, target, and completion-time equations in the main text.

Table 5 compares five algebraic methods on nine process GDP models. Of the 45 runs, 43 return solutions that pass the original-GDP residual test at 10^{-5}. Big-M returns no accepted feasible solution for `CSTR15` or `Column` within the time limit. The target-attainment counts are 6/9 for RIC, 1/9 for GLOA, 6/9 for LOA, 5/9 for Big-M, and 7/9 for Hull. These single-run results do not establish success probabilities or global optimality.

All five methods attain the `Small Batch` reference. Big-M and Hull complete in 0.51 and 0.50 s, respectively, compared with median DFO completion estimates ranging from 3.49 to 9.09 s. This demonstrates the competitiveness of algebraic methods when the formulation is available, although the different information access and timing protocols prevent direct speedup claims.

All six `CSTR` GLOA runs report `optimal` but miss the lookup references. Their reported lower bounds exceed independently checked feasible objectives. The `Dynamic` Hull run has the same inconsistency: its lower bound is -196.931, while the saved Big-M point has objective -212.009. That point also passes a check against the reconstructed Hull formulation (Supplementary Section S7). The uniform bound check flags 19 results, including these seven large discrepancies and 12 smaller exceedances of 1.09-4.30 times the comparison tolerance. The flags distinguish numerical consistency from target attainment and reported termination; they do not establish exact feasibility or a solver defect.

A separate `CSTR15` check on Dantzig using `IPOPT` 3.14.19 retained the archived LOA witness W03's discrete configuration and starting values (Supplementary Section S7) (SECQUOIA, 2026). The returned objective was 2.82939, with the maximum unscaled original-GDP violation 1.11×10^{-16}, passing a residual target 10^{-8}. The current `SCIP`-based GLOA lower bound exceeds this previously validated objective by 0.732507, approximately 2.59×10^4 times the audit tolerance. The discrepancy therefore persists against the tighter feasibility check, although its cause remains unresolved.

Table 5: Full-process-GDP results: objective (measured completion seconds, including model-template preparation). `GDPOpt` uses `SCIP` for master problems and nonlinear subproblems; Big-M and Hull also use `SCIP`. Feasibility means numerical feasibility under the original-GDP residual test at 10^{-5}, not exact feasibility. D: Why are these results in a table and not merged inside figures 3 and 4? They also have completion times and NLP iterations (actually let's call the horizontal axis as NLP problems solved A: Each algebraic method was run once per problem, whereas these figures summarize ten DFO seeds per condition. The evaluation counts are not directly comparable: DFO configuration evaluations include preprocessing rejections without an NLP solve. DFO completion times are reconstructed estimates, while algebraic runtimes are measured directly. I therefore kept the algebraic results in a separate table.

Model	RIC	GLOA	LOA	Big-M	Hull
`CSTR5`	$3.06201^{\dagger}$ (25.75)	$3.13019^{\dagger\ddagger}$ (5.68)	$3.06201^{\dagger}$ (25.66)	3.06198 (53.68)	$3.06201^{\dagger}$ (15.43)
`CSTR10`	$2.88954^{\dagger}$ (215.05)	$3.44794^{\dagger\ddagger}$ (6.36)	$2.88954^{\dagger}$ (234.45)	2.88941 (771.36)	$2.88951^{\dagger}$ (71.63)
`CSTR15`	$2.82939^{\dagger}$ (326.02)	$3.56190^{\dagger\ddagger}$ (7.11)	$2.82939^{\dagger}$ (545.18)	$-^{\mathrm{TL}}$ (1790.95)	2.82936 (284.63)
`CSTR20`	$2.79941^{\dagger}$ (766.97)	$3.61603^{\dagger\ddagger}$ (9.17)	$2.79941^{\dagger}$ (1018.28)	2.80413^{TL} (1792.44)	2.79936 (1150.67)
`CSTR25`	$2.78148^{\dagger}$ (1391.67)	$3.64775^{\dagger\ddagger}$ (11.22)	$2.78148^{\dagger}$ (1460.66)	2.78771^{TL} (1792.33)	2.78143 (1701.75)
`CSTR30`	2.80939^{TL} (1800.09)	$3.66862^{\dagger\ddagger}$ (14.48)	2.81527^{TL} (1789.88)	2.76957^{TL} (1791.82)	2.76957^{TL} (1792.56)
`Column`	22271.5^{TL} (1791.52)	22271.5^{TL} (1791.51)	22271.5^{TL} (1791.66)	$-^{\mathrm{TL}}$ (1792.83)	22271.5^{TL} (1792.76)
`Dynamic`	-117.074^{TL} (1791.57)	-117.074^{TL} (1791.46)	-117.074^{TL} (1791.46)	-212.009^{TL} (1791.75)	$-196.931^{\dagger\ddagger}$ (921.28)
`Small Batch`	167428 (13.33)	167428 (10.92)	167428 (47.77)	167428 (0.51)	167428 (0.50)

Raw termination is `optimal` unless TL denotes `maxTimeLimit`. A dash denotes no accepted feasible solution. Reference-target classification uses the lookup-reference values in Table 1. ‡: reference target not attained despite the reported `optimal`; a better independently checked numerically feasible solution exists under the stated residual test, so the report is not treated as an independent optimality certificate. This is a target miss, not an execution failure. †: reported lower bound exceeds the best checked saved numerically feasible value f by more than $10^{-5}\max\{1, |f|\}$; the bound-consistency results and comparison solutions are summarized in Supplementary Section S7, Tables S5-S6, and the `SCIP` analysis files linked there. Near-tolerance cases are discussed in the text. The same rule applies to all methods. The 43 saved feasible vectors pass the original-GDP recheck at 10^{-5}, which does not establish optimality. Each method/model has one run; no success probability is inferred. Times are full-run completion times.

4.4 Integrated Numerical Discussion

The results support RBFOpt for configuration search under limited evaluation budgets, particularly on CSTR and at low budgets on Dynamic. Its stronger Grid attainment is consistent with this result, while Column and Small Batch show that the advantage is problem-dependent.

D: why are you so fixated on RBFOpt? BAM shows even better results! Get back to the main story of the paper and have the sections support it

Target attainment must be weighed against search overhead and stopping behavior. For Dynamic at $B = 30$, D-SDA uses a median of 23.5 callbacks, compared with 30 for RBFOpt. Their median post-target counts are 6.5 and 23, respectively. Thus full-run completion costs include work after the target is first observed.

These findings concern the tested configuration domains and fixed local-solver responses. The simultaneous changes in CSTR size and budget do not isolate scalability.

Algebraic formulations provide competitive reference results on Grid-GDP and Small Batch. Their access to model structure differs from that of DFO, and the inconsistent bounds discussed in Section 4.3 cannot serve as independent certificates.

5 Conclusions

This study evaluated RBFOpt for GDP configuration search using external integer variables. It reached 45 of 60 CSTR targets under the tested budgets, compared with 14 for D-SDA. On Dynamic, it reached all ten targets at $B = 12$, while D-SDA first did so at $B = 30$. Across the Grid-GDP and process GDP benchmarks, higher target coverage did not consistently yield lower estimated full-run completion times. Performance depends on the problem, failure responses, and search overhead. D: Absolutely not, these are not the study conclusions

These results support RBFOpt as an alternative for GDP configuration search under limited evaluation budgets. D-SDA performed better on Small Batch. Algebraic approaches remain useful alternatives when model expressions and suitable solvers are available.

The findings are conditional on finite candidate domains and frozen local-solver responses. Completion costs are retrospective estimates, not measured online optimizer-NLP times. The conflicting algebraic optimality reports remain unresolved. Transfer to broader configuration spaces and live NLP evaluation remain to be tested.

6 Acknowledgement

The authors acknowledge and express sincere gratitude to the Davidson School of Chemical Engineering at Purdue University for providing a supportive research environment and funding.

Data and code availability

The code and data are available in the Black-Box-for-GDP repository (SECQUOIA, 2026). Supplementary Section S6 identifies the archived inputs, including the CSTR updates, Column logical-domain runs, and SCIP algebraic comparisons.

References

Amestoy, P.R., Duff, I.S., Koster, J., L'Excellent, J.Y., 2001. A fully asynchronous multifrontal solver using distributed dynamic scheduling. SIAM Journal on Matrix Analysis and Applications 23, 15–41. doi:10.1137/S0895479899358194.

Bergamini, M.L., Aguirre, P., Grossmann, I., 2005. Logic-based outer approximation for globally optimal synthesis of process networks. Comput. Chem. Eng. 29, 1914–1933. doi:10.1016/j.compchemeng.2005.04.003.

Bolusani, S., Besançon, M., Bestuzheva, K., Chmiela, A., Dionísio, J., Donkiewicz, T., van Doornmalen, J., Eifler, L., Ghannam, M., Gleixner, A., Graczyk, C., Halbig, K., Hedtke, I., Hoen, A., Hojny, C., van der Hulst, R., Kamp, D., Koch, T., Kofler, K., Lentz, J., Manns, J., Mexi, G., Mühmer, E., Pfetsch, M.E., Schlösser, F., Serrano, F., Shinano, Y., Turner, M., Vigerske, S., Weninger, D., Xu, L., 2024. The SCIP Optimization Suite 9.0. ZIB Report 24-02-29. Zuse Institute Berlin. URL: https://nbn-resolving.org/urn:nbn:de:0297-zib-95528.

Bonami, P., Biegler, L.T., Conn, A.R., Cornuéjols, G., Grossmann, I.E., Laird, C.D., Lee, J., Lodi, A., Margot, F., Sawaya, N., Wächter, A., 2008. An algorithmic framework for convex mixed integer nonlinear programs. Discrete Optimization 5, 186–204. doi:10.1016/j.disopt.2006.10.011.

Bynum, M.L., Hackebeil, G.A., Hart, W.E., Laird, C.D., Nicholson, B.L., Siirola, J.D., Watson, J.P., Woodruff, D.L., 2021. Pyomo: Optimization Modeling in Python. volume 67 of *Springer Optimization and Its Applications*. 3 ed., Springer. doi:10.1007/978-3-030-68928-5.

Chen, Q., Johnson, E.S., Bernal, D.E., Valentin, R., Kale, S., Bates, J., Siirola, J.D., Grossmann, I.E., 2022. Pyomo.GDP: an ecosystem for logic based modeling and optimization development. Optim. Eng. 23, 607–642. doi:10.1007/s11081-021-09601-7.

Clopper, C.J., Pearson, E.S., 1934. The use of confidence or fiducial limits illustrated in the case of the binomial. Biometrika 26, 404–413. doi:10.1093/biomet/26.4.404.

Costa, A., Nannicini, G., 2018. RBFOpt: an open-source library for black-box optimization with costly function evaluations. Math. Program. Comput. 10, 597–629. doi:10.1007/s12532-018-0144-7.

Cozad, A., Sahinidis, N.V., Miller, D.C., 2014. Learning surrogate models for simulation-based optimization. AIChE J. 60, 2211–2227. doi:10.1002/aic.14418.

D-Wave Systems, n.d. dwave-samplers: Classical algorithms for solving binary quadratic models. URL: https://github.com/dwavesystems/dwave-samplers/releases/tag/1.6.0. Version 1.6.0; classical CPU simulated annealing; accessed October 15, 2026.

Dolan, E.D., Moré, J.J., 2002. Benchmarking optimization software with performance profiles. Math. Program. 91, 201–213. doi:10.1007/s101070100263.

GAMS Development Corporation, 2025. General Algebraic Modeling System (GAMS). URL: https://www.gams.com/. release 52.2.0, Fairfax, VA, USA.

Ghouse, J.H., Chen, Q., Zamarripa, M.A., Lee, A., Burgard, A.P., Grossmann, I.E., Miller, D.C., 2018. A comparative study between gdp and nlp formulations for conceptual design of distillation columns, in: Computer Aided Chemical Engineering, pp. 865–870. doi:10.1016/B978-0-444-64241-7.50139-7.

Grossmann, I.E., Lee, S., 2003. Generalized convex disjunctive programming: Nonlinear convex hull relaxation. Comput. Optim. Appl. 26, 83–100. doi:10.1023/A:1025154322278.

Kingma, D.P., Ba, J., 2015. Adam: A method for stochastic optimization. International Conference on Learning Representations (ICLR). URL: https://arxiv.org/abs/1412.6980.

Kirkpatrick, S., Gelatt, Jr., C.D., Vecchi, M.P., 1983. Optimization by simulated annealing. Science 220, 671–680. doi:10.1126/science.220.4598.671.

Kitai, K., Guo, J., Ju, S., Tanaka, S., Tsuda, K., Shiomi, J., Tamura, R., 2020. Designing metamaterials with quantum annealing and factorization machines. Phys. Rev. Res. 2, 013319. doi:10.1103/PhysRevResearch.2.013319.

Kocis, G.R., Grossmann, I.E., 1988. Global optimization of nonconvex mixed-integer nonlinear programming (minlp) problems in process synthesis. Industrial & Engineering Chemistry Research 27, 1407–1421. doi:10.1021/ie00080a013.

Larson, J., Menickelly, M., Wild, S.M., 2019. Derivative-free optimization methods. Acta Numerica 28, 287–404. doi:10.1017/S0962492919000060.

Liñán, D.A., Bernal, D.E., Ricardez-Sandoval, L.A., Gómez, J.M., 2020. Optimal design of superstructures for placing units and streams with multiple and ordered available locations. part i: A new mathematical framework. Computers & Chemical Engineering 137, 106794. doi:10.1016/j.compchemeng.2020.106794.

Ma, K., Rios, L.M., Bhosekar, A., Sahinidis, N.V., Rajagopalan, S., 2023. Branch-and-Model: a derivative-free global optimization algorithm. Comput. Optim. Appl. 85, 337–367. doi:10.1007/s10589-023-00466-3.

Moré, J.J., Wild, S.M., 2009. Benchmarking derivative-free optimization algorithms. SIAM J. Optim. 20, 172–191. doi:10.1137/080724083.

Nannicini, G., 2021. On the implementation of a global optimization method for mixed-variable problems. Open J. Math. Optim. 2, 1–25. doi:10.5802/ojmo.3.

Ovalle, D., Liñán, D.A., Lee, A., Gómez, J.M., Ricardez-Sandoval, L., Grossmann, I.E., Bernal Neira, D.E., 2025. Logic-Based Discrete-Steepest Descent: A solution method for process synthesis Generalized Disjunctive Programs. Comput. Chem. Eng. 195, 108993. doi:10.1016/j.compchemeng.2024.108993.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., Chintala, S., 2019. PyTorch: An imperative style, high-performance deep learning library, in: Advances in Neural Information Processing Systems, pp. 8024–8035.

Peng, Z., Lee, A., Bernal Neira, D.E., 2024. Addressing discrete dynamic optimization via a logic-based discrete-steepest descent algorithm, in: 2024 IEEE 63rd Conference on Decision and Control (CDC), IEEE. pp. 1664–1669. doi:10.1109/CDC56724.2024.10886477.

Pyomo Development Team, n.d. GDPopt logic-based solver. URL: https://pyomo.readthedocs.io/en/6.10.0/explanation/solvers/gdpopt.html. Version 6.10.0; accessed October 20, 2026.

Raman, R., Grossmann, I.E., 1994. Modelling and computational techniques for logic based integer programming. Comput. Chem. Eng. 18, 563–578. doi:10.1016/0098-1354(93)E0010-7.

Regis, R.G., Shoemaker, C.A., 2007. A stochastic radial basis function method for the global optimization of expensive functions. INFORMS J. Comput. 19, 497–509. doi:10.1287/ijoc.1060.0182.

Rendle, S., 2010. Factorization machines, in: 2010 IEEE International Conference on Data Mining, pp. 995–1000. doi:10.1109/ICDM.2010.127.

Sahinidis, N.V., 1996. BARON: A general purpose global optimization software package. J. Glob. Optim. 8, 201–205. doi:10.1007/BF00138693.

SECQUOIA, 2026. Black-Box-for-GDP: benchmark code, experimental data, and analysis. GitHub. URL: https://github.com/SECQUOIA/Black-Box-for-GDP/tree/49d4bcbeb10fe49b74fe2333d27829046e93bf03. commit 49d4bcbe; accessed 28 October 2026.

SECQUOIA, n.d. bbo_via_fmqa: Black-box optimization via factorization machines and quantum annealing. URL: https://github.com/SECQUOIA/bbo_via_fmqa/tree/eb3a20d4b5a3773d17ff31d72a35d51f58e84e56. QHD_Data branch; commit eb3a20d4b5a3773d17ff31d72a35d51f58e84e56; upstream PyTorch FM and BQM code; accessed October 19, 2026.

The Optimization Firm, 2026. BAM python interface documentation: bam-solver 5.0.0. URL: https://pypi.org/project/bam-solver/5.0.0/. Version 5.0.0, released September 11, 2026; executed engine 2026.9.10; accessed October 20, 2026.

Türkay, M., Grossmann, I.E., 1996. Logic-based MINLP algorithms for the optimal synthesis of process networks. Comput. Chem. Eng. 20, 959–978. doi:10.1016/0098-1354(95)00219-7.

Wächter, A., Biegler, L.T., 2006. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. Mathematical Programming 106, 25–57. doi:10.1007/s10107-004-0559-y.

Wu Group, n.d. Quantum Hamiltonian descent: 2D test functions. https://jiaqileng.github.io/quantum-hamiltonian-descent/nonconvex2d.html. Source collection for the 21 adapted finite grids used here; accessed September 29, 2026.

Supplementary Material: Benchmark Definitions, Execution Protocol, and Uncertainty

The Grid-GDP functions are adapted from the Quantum Hamiltonian Descent collection; each instance is defined by its implemented expression and domain rather than its conventional name (SECQUOIA, 2026; Wu Group, n.d.).

S1 Coordinate domains and normalization

For each row of Table S1, both endpoints are included and the physical coordinates are

$$a_i = \ell_x + \frac{i}{30}(u_x - \ell_x), \qquad b_j = \ell_y + \frac{j}{30}(u_y - \ell_y), \qquad i, j \in \{0, \ldots, 30\}. \tag{S1}$$

For the raw functions in Section S2, the minimized objective is

$$F(x, y) = \frac{f(x, y) - L}{U - L}, \qquad L = \min_{i,j} f(a_i, b_j), \quad U = \max_{i,j} f(a_i, b_j). \tag{S2}$$

Thus L and U are obtained by exhaustive preprocessing of the 961 lattice points, not from the continuous optimum. All trigonometric arguments are in radians. Full-precision normalization constants and target coordinates are archived in `grid_inventory.json` (Section S6).

Table S1: Grid-GDP benchmark domains and target-set sizes. Each problem uses a 31×31 lattice with 961 configurations and 62 Boolean indicators. n_{target} counts lattice points with normalized objective $F \leq 10^{-5}$.

Function	x domain	y domain	n_{target}
Ackley	$[-5, 5]$	$[-5, 5]$	1
Alpine 1	$[0, 10]$	$[0, 10]$	1
Alpine 2	$[0, 1000]$	$[0, 1000]$	1
Bohachevsky 2	$[-100, 100]$	$[-100, 100]$	1
Three-hump camel	$[-5, 5]$	$[-5, 5]$	1
Csendes (squared variant)	$[-1, 1]$	$[-1, 1]$	36
Deflected corrugated spring	$[0, 10]$	$[0, 10]$	1
Drop-Wave	$[-5.12, 5.12]$	$[-5.12, 5.12]$	1
Easom	$[-10, 10]$	$[-10, 10]$	1
Griewank	$[-600, 600]$	$[-600, 600]$	1
Hölder table	$[-10, 10]$	$[-10, 10]$	4
Hosaki	$[0, 5]$	$[0, 6]$	1
Lévy	$[-10, 10]$	$[-10, 10]$	2
Lévy 13	$[-10, 10]$	$[-10, 10]$	4
Michalewicz	$[0, \pi]$	$[0, \pi]$	1
Rastrigin	$[-5.12, 5.12]$	$[-5.12, 5.12]$	1
Rosenbrock	$[-2, 2]$	$[-1, 3]$	2
Shubert (historical variant)	$[0, 10]$	$[0, 10]$	1
Styblinski–Tang	$[-5, 5]$	$[-5, 5]$	1
Sum of squares	$[-10, 10]$	$[-10, 10]$	1
Xin-She Yang 3 (historical variant)	$[-20, 20]$	$[-20, 20]$	870

S2 Grid-GDP objective functions

The implemented functions below use the domains in Table S1 (SECQUOIA, 2026).

1. Ackley

$$f(x,y) = -20\exp\left(-0.2\sqrt{\frac{x^2+y^2}{2}}\right) - \exp\left(\frac{\cos(2\pi x)+\cos(2\pi y)}{2}\right) + 20 + e. \tag{S3}$$

2. Alpine 1

$$f(x,y) = |x\sin x + 0.1x| + |y\sin y + 0.1y|. \tag{S4}$$

3. Alpine 2

$$f(x,y) = -\sqrt{xy}\,\sin x \sin y. \tag{S5}$$

4. Bohachevsky 2

$$f(x,y) = x^2 + 2y^2 - 0.3\cos(3\pi x)\cos(4\pi y) + 0.3. \tag{S6}$$

5. Three-hump camel

$$f(x,y) = 2x^2 - 1.05x^4 + \frac{x^6}{6} + xy + y^2. \tag{S7}$$

6. Csendes (squared variant)

$$f(x,y) = c(x) + c(y), \qquad c(t) = \begin{cases} t^6\left[2+\sin(1/t)\right]^2, & t \neq 0, \\ 0, & t = 0. \end{cases} \tag{S8}$$

The GDP uses branch-fixed auxiliaries $r_t = 1/t$ for $t \neq 0$ and $r_t = 0$ otherwise, avoiding division by zero without an epsilon shift.

7. Deflected corrugated spring

With $r = \sqrt{(x-5)^2 + (y-5)^2}$,

$$f(x,y) = 0.1r^2 - \cos(5r). \tag{S9}$$

8. Drop-Wave

$$f(x,y) = -\frac{1+\cos\left(12\sqrt{x^2+y^2}\right)}{0.5(x^2+y^2)+2}. \tag{S10}$$

9. Easom

$$f(x,y) = -\cos x \cos y\, \exp\left(-[(x-\pi)^2 + (y-\pi)^2]\right). \tag{S11}$$

10. Griewank

$$f(x,y) = \frac{x^2+y^2}{4000} - \cos x \cos\left(\frac{y}{\sqrt{2}}\right) + 1. \tag{S12}$$

11. Hölder table

$$f(x,y) = -\left|\sin x \cos y\, \exp\left(\left|1 - \frac{\sqrt{x^2+y^2}}{\pi}\right|\right)\right|. \tag{S13}$$

12. Hosaki

$$f(x,y) = \left(1 - 8x + 7x^2 - \frac{7}{3}x^3 + \frac{1}{4}x^4\right) y^2 \exp(-y). \tag{S14}$$

13. Lévy

Define $w_x = 1 + (x-1)/4$ and $w_y = 1 + (y-1)/4$. Then

$$\begin{aligned} f(x,y) = {} & \sin^2(\pi w_x) \\ & + (w_x - 1)^2\big[1 + 10\sin^2(\pi w_x + 1)\big] \\ & + (w_y - 1)^2\big[1 + \sin^2(2\pi w_y)\big]. \end{aligned} \tag{S15}$$

14. Lévy 13

$$\begin{aligned} f(x,y) = {} & \sin^2(3\pi x) + (x-1)^2\big[1 + \sin^2(3\pi y)\big] \\ & + (y-1)^2\big[1 + \sin^2(2\pi y)\big]. \end{aligned} \tag{S16}$$

15. Michalewicz

$$f(x,y) = -\sin x\, \sin^{20}\left(\frac{x^2}{\pi}\right) - \sin y\, \sin^{20}\left(\frac{2y^2}{\pi}\right). \tag{S17}$$

16. Rastrigin

$$f(x,y) = x^2 - 10\cos(2\pi x) + y^2 - 10\cos(2\pi y) + 20. \tag{S18}$$

17. Rosenbrock

$$f(x,y) = (1-x)^2 + 100(y - x^2)^2. \tag{S19}$$

18. Shubert (historical variant)

$$\begin{aligned} f(x,y) = {} & \big[\cos(2x+1) + 2\cos(3x+2) + 3\cos(4x+3)\big] \\ & \times \big[\cos(2y+1) + \cos(y+2)\big]. \end{aligned} \tag{S20}$$

19. Styblinski–Tang

$$f(x,y) = \frac{1}{78}\,\frac{1}{2}\left(x^4 - 16x^2 + 5x + y^4 - 16y^2 + 5y\right). \tag{S21}$$

20. Sum of squares

$$f(x,y) = x^2 + 2y^2. \tag{S22}$$

21. Xin-She Yang 3 (historical variant)

$$f(x,y) = \exp\left(-\left[\left(\frac{x}{15}\right)^6 + y^6\right]\right) + 2\exp\big(-(x^2+y^2)\big)\cos^2 x \cos^2 y. \tag{S23}$$

S3 Executable protocol and implementation settings

Python and NumPy use the job seed, and `RBFOpt` also receives `rand_seed`. `BAM` exposes no separate native random-seed option: its process wrapper seeds Python/NumPy, while the Grid seed identifies the common initial design. `FMQA-SA` uses three NumPy `SeedSequence(seed)` streams for FM initialization, annealing, and uniform unseen-point fallback; each training or annealing invocation draws a seed from its stream. `PyTorch` uses deterministic CPU operations and one thread.

Grid `D-SDA` enumerates L_∞ neighbors lexicographically from $(-1, 0, 1)$, excluding the zero direction and out-of-bounds points; ties select the first point.

The `RBFOpt` options are `rbf=auto`, `global_search_method=genetic`, `do_init_strategy=False`, `ga_base_population_size=400`, and `ga_num_generations=20`. `RBFOpt` 4.3.0 uses an effective population of $400 + \lfloor 20n/5 \rfloor$, where n is the dimension passed to the genetic-search routine, giving 408 and 412 for two and three search variables. `BAM` receives `x0=None` and `max_evals`=B-3; its local models use `ALAMO`/`BARON`, whose internal versions are not reported by the package.

The `FMQA-SA` time limit is checked between operations and through training/sampling callbacks. Each FM uses identity activation, zero bias, and normal initial weights with mean zero and standard deviation 0.01. Training uses Adam (Kingma and Ba, 2015), 1,000 full-batch epochs, mean squared error, and learning rate 0.01. Coordinate ξ_j uses $\lfloor \log_2 u_j \rfloor + 1$ unsigned bits, where $u_j > 0$ is its upper bound; QUBO coefficients are divided by the largest absolute entry of the quadratic matrix. D-Wave's `SimulatedAnnealingSampler` in `dwave-samplers` 1.6.0 uses 100 reads, 1,000 sweeps, and a geometric inverse-temperature schedule (D-Wave Systems, n.d.). All process workers and the Grid `FMQA-SA` worker have a 2,100 s watchdog. The Grid `RBFOpt`/`BAM` trace watchdogs are disabled (`max_run_seconds=0`); archived Grid `D-SDA` uses budget, target, or local-search termination without an additional method-clock limit.

S3.1 Versions and independently timed process evaluations

The recorded Python version is 3.11.16; other versions are given in the main paper (Section 3.4). The recorded Grid `GDPOpt` runs make no NLP solver calls because preprocessing fixes all original variables. The algebraic process models retain their archived initializations, including constructor defaults for `CSTR30`; the updated DFO evaluation table is identified separately in Section S6.

The acceptance rule in `gdp_models/evaluation.py` requires all of the following: a finite objective; termination in {`optimal`, `locallyOptimal`, `globallyOptimal`}; solver status other than `error` or `aborted`; interface status `SOLVESTAT`=1 and `MODELSTAT`$\in \{1, 2\}$; and a finite maximum primal violation no greater than 10^{-5}. Time-limit, iteration-limit, evaluation-limit, and error outcomes are not accepted, even if a finite feasible candidate is available.

The primal check uses unscaled violations of original constraints and initialized-variable bounds, including constraint rows deactivated during preprocessing. Nonfinite objectives, constraint bodies, or initialized-variable values preclude acceptance; unused uninitialized variables are skipped. Preprocessing rejection or worker failure supplies no accepted objective.

The `CSTR` initializer `cstr_box_ipopt.py:initialize_feasible` runs after preprocessing and checks the mass-balance/zero-recycle start at 10^{-8} without changing constraints or bounds. All process evaluations use cold starts; NLP solutions are not transferred between configurations. Objectives, acceptance status, and NLP point costs are linked to the same recorded attempt by model, `CSTR` size, and coordinate, without successful-retry selection or cross-size time substitution.

The CSV is loaded before timing starts; callback logging remains in search time rather than the subtracted lookup time.

S3.2 Observed proposal and initialization behavior

Table S2 pools `FMQA-SA` proposals across budgets and seeds, excluding the shared initialization and including terminal callbacks. Fallback supplies 52.89% of process proposals; the uniform comparison evaluates the complete procedure, without isolating the contributions of learning, annealing, and fallback.

Table S2: Recorded `FMQA-SA` uniform fallback by benchmark family.

Family	Runs	Proposals	Fallbacks	Rate
Grid	1,050	51,450	9,522	18.51%
`CSTR`	60	1,350	369	27.33%
`Column`	10	100	86	86.00%
`Dynamic`	60	1,460	1,063	72.81%
`Small Batch`	10	70	58	82.86%
Process total	140	2,980	1,576	52.89%

S4 Process model scale, units, and discrete decisions

Table S3 counts the original GDP before reformulation or presolve, including all active disjunct branches and fixed continuous variables. `Dynamic` is discretized with 30 finite elements per stage and three Radau points. Binary counts refer to the indicator variables already constructed by the model, before Boolean-to-binary conversion. All nine models have zero additional nonbinary integer variables.

Table S3: Original process-model sizes. n_c: continuous variables; n_b: binary variables; m_a: algebraic constraints; m_l: logical constraints; n_d: disjunctions.

Model	n_c	n_b	m_a	m_l	n_d
`CSTR5`	56	20	100	12	10
`CSTR10`	106	40	195	22	20
`CSTR15`	156	60	290	32	30
`CSTR20`	206	80	385	42	40
`CSTR25`	256	100	480	52	50
`CSTR30`	306	120	575	62	60
`Column`	446	30	637	16	15
`Small Batch`	19	18	34	3	9
`Dynamic`	1647	27	3276	31	9

`CSTR` volume is measured in litres, and `Small Batch` investment uses the source model's dollars without currency-year conversion. The `Column` objective is $10^3(Q_c + Q_b) + 10^3 n_{\text{active}}$, the source model's weighted cost proxy rather than a newly estimated annualized cost. The `Dynamic` objective is $-\int_0^9 x(t)^2\,dt$, with no physical-unit calibration for state or time.

S5 Success-rate uncertainty and completion costs

For each problem, method, and budget, we report target hits h out of ten seeds with pointwise two-sided 95% Clopper–Pearson intervals $[p_L, p_U]$, counting execution failures as non-successes (Clopper and Pearson, 1934). Coverage assumes independent Bernoulli outcomes under the fixed protocol; it is not simultaneous across comparisons. Ten successes give $[0.692, 1]$, and zero give $[0, 0.308]$, so neither observed endpoint establishes a population probability of one or zero. Table S4 reports completion costs for all 14 process conditions.

Per-function Grid results at $B = 100$ and complete seed-level outcomes are available in the analysis files linked in Section S6.

Table S4: Median reconstructed full-run completion costs for all ten paired seeds per condition, including target misses, in seconds.

Case	B	D-SDA	RBFOpt	BAM	FMQA-SA	Uniform
Column	13	35.44	54.60	27.50	41.82	21.69
CSTR5	8	10.67	10.72	10.67	13.92	9.44
CSTR10	15	18.95	21.79	18.54	27.07	17.75
CSTR15	22	29.93	37.59	27.67	42.82	27.84
CSTR20	29	40.68	45.03	39.37	59.10	40.12
CSTR25	36	51.69	62.29	50.47	74.70	51.17
CSTR30	43	53.12	101.96	115.41	101.19	65.20
Small Batch	10	3.84	5.12	4.23	9.09	3.49
Dynamic	12	12.23	12.29	9.95	14.47	7.80
Dynamic	15	15.85	16.10	11.53	18.39	8.71
Dynamic	20	21.30	20.54	14.07	25.23	12.27
Dynamic	30	26.62	34.19	21.91	36.24	19.38
Dynamic	37	26.62	42.42	24.99	46.05	24.38
Dynamic	50	26.62	65.14	33.98	61.31	33.12

These are full-run estimates, not first-hit times.

S6 Data sources and reproducibility

The publication snapshot contains the source code, experimental data, analysis results, and reproduction instructions for the DFO and algebraic comparisons. The Grid `GDPOpt` summary combines original non-error runs with recovery runs for the 46 execution errors, so its counts do not establish a method ranking. The three presolve-disabled LOA diagnostic runs are excluded.

S7 Feasibility and bound consistency of the algebraic results

Of the 45 algebraic runs reported in the main paper (Table 5), 19 have flagged bounds (Table S5) relative to the checked solutions in Table S6. Unrounded values, residuals, and source records are linked in Section S6.

For each minimization model, let f_w be the recorded objective of its best independently checked saved feasible solution, and let L denote a reported lower bound. We apply the same rule to all five methods:

$$\tau_w = 10^{-5}\max\{1, |f_w|\}, \qquad \Delta = L - f_w, \qquad \rho = \Delta/\tau_w, \qquad \dagger \iff \rho > 1. \tag{S24}$$

Missing lower bounds remain missing, not zero.

Saved vectors are loaded into the original GDP to reevaluate objectives and check active constraints, variable bounds, integrality, disjunctions, and logical validity without optimization. Violations are unscaled original-equation residuals, not reformulation residuals. All 43 saved feasible vectors satisfy the 10^{-5} threshold; the largest violation is 9.99×10^{-7}. The two Big-M runs without accepted feasible solutions remain in the 45-run denominator. This numerical check establishes neither exact feasibility nor global optimality.

Twelve of the 19 discrepancies are only 1.09–4.30 times the tolerance and are sensitive to numerical feasibility conventions. These conflicting bounds and `optimal` reports provide no independent global certificate or intrinsic method ranking; their cause in GDPopt or its subsolvers remains unresolved.

Table S5: Flagged bound comparisons, using unrounded $\rho > 1$. C identifies the comparison solution in Table S6; displayed values are rounded. Flags indicate numerical inconsistencies of different magnitudes.

ID	Model	Method	Reported L	$v_{\max}$	C	ρ
S01	`CSTR5`	RIC	3.06201	9.31e-7	C01	1.13†
S02	`CSTR5`	GLOA	3.13019	9.99e-7	C01	2.23e+3†
S03	`CSTR5`	LOA	3.06201	9.31e-7	C01	1.13†
S05	`CSTR5`	Hull	3.06201	9.34e-7	C01	1.09†
S06	`CSTR10`	RIC	2.88954	7.87e-11	C02	4.30†
S07	`CSTR10`	GLOA	3.44794	9.16e-7	C02	1.93e+4†
S08	`CSTR10`	LOA	2.88954	7.87e-11	C02	4.30†
S10	`CSTR10`	Hull	2.88951	9.37e-7	C02	3.54†
S11	`CSTR15`	RIC	2.82939	1.39e-9	C03	1.30†
S12	`CSTR15`	GLOA	3.56190	9.78e-7	C03	2.59e+4†
S13	`CSTR15`	LOA	2.82939	1.39e-9	C03	1.30†
S16	`CSTR20`	RIC	2.79941	4.47e-9	C04	1.67†
S17	`CSTR20`	GLOA	3.61603	8.93e-7	C04	2.92e+4†
S18	`CSTR20`	LOA	2.79941	4.47e-9	C04	1.67†
S21	`CSTR25`	RIC	2.78148	9.72e-11	C05	1.72†
S22	`CSTR25`	GLOA	3.64775	9.83e-7	C05	3.11e+4†
S23	`CSTR25`	LOA	2.78148	9.72e-11	C05	1.72†
S27	`CSTR30`	GLOA	3.66862	9.40e-7	C06	3.25e+4†
S45	`Dynamic`	Hull	-196.931	1.00e-7	C09	7.11e+3†

Additional tightened-feasibility check for `CSTR15`. A Dantzig `IPOPT` 3.14.19 solve retained the archived LOA witness W03's discrete configuration and starting values and targeted an original-GDP

Table S6: Checked saved comparison solutions; displayed values are rounded.

ID	Model	Method	Recorded f_w	$v_{\max}$
C01	`CSTR5`	Big-M	3.06198	9.40e-7
C02	`CSTR10`	Big-M	2.88941	9.76e-7
C03	`CSTR15`	Hull	2.82936	9.47e-7
C04	`CSTR20`	Hull	2.79936	9.90e-7
C05	`CSTR25`	Hull	2.78143	9.90e-7
C06	`CSTR30`	Big-M	2.76957	9.68e-9
C07	`Column`	GLOA	22271.5	1.00e-8
C08	`Small Batch`	GLOA	167428	9.00e-7
C09	`Dynamic`	Big-M	-212.009	1.00e-7

residual of 10^{-8} (SECQUOIA, 2026). W03 is distinct from the current `SCIP` comparison solution C03. Only exactly satisfied constant rows were omitted from the computational clone; the independent check retained all original constraints. The returned objective was 2.82939, with maximum unscaled violation 1.11×10^{-16}. The current `SCIP` GLOA lower bound exceeds this objective by 0.732507, or approximately 2.59×10^4 times the audit tolerance. The discrepancy persists under the tighter check, but its cause remains unresolved.

Additional formulation check for `Dynamic` Hull. The saved Big-M point has objective -212.009, approximately 15.0776 below the Hull lower bound -196.931. It satisfies both the original GDP and a reconstructed Hull formulation at 10^{-5}, with maximum violation approximately 10^{-7} after assigning the auxiliary variables from their definitions. The reconstructed export matches the archived preflight export, but the compiled model from the actual solve was not retained. The check therefore confirms the numerical inconsistency without identifying its solver or interface cause; no optimization was rerun.

S8 Additional candidate-search and sampling comparisons

S8.1 Paired differences and sampling controls

Paired success differences retain the common case, budget and seed. For Grid, the descriptive percentile bootstrap resamples the 21 intact function vectors, each containing ten paired seed differences, rather than treating 210 observations as exchangeable Bernoulli draws. For Process, paired seeds are resampled within each of the 14 conditions; the six related `Dynamic` budgets are not pooled for inference. The seed is 20261020 with 2,000 bootstrap replications and deterministic case/seed ordering. Intervals are exploratory and unadjusted for multiple comparisons; ten-seed boundary cases may yield degenerate intervals, and zero inclusion does not establish equivalence.

Table S7: Selected paired success differences, in percentage points. The first method minus the second defines the sign; Process intervals concern the stated condition only.

Grid

B	Contrast	Difference	95% interval
10	`RBFOpt` – Uniform	13.33	[5.71, 24.29]
100	`RBFOpt` – Uniform	60.00	[42.86, 75.71]
10	`BAM` – `BAM`-prefix control	0.00	[0.00, 0.00]
100	`BAM` – `BAM`-prefix control	10.95	[0.48, 23.81]
10	`FMQA-SA` – Uniform	0.95	[-0.95, 2.86]
100	`FMQA-SA` – Uniform	40.00	[28.57, 52.38]
10	`D-SDA` – `RBFOpt`	-13.81	[-24.29, -6.19]
100	`D-SDA` – `RBFOpt`	-33.33	[-48.10, -19.52]

Process: ten paired seeds per condition

Case	B	Contrast	Outcomes	Difference	95% interval
`Dynamic`	12	`RBFOpt` – `D-SDA`	4/6/0/0	60	[30, 90]
`Small Batch`	10	`FMQA-SA` – Uniform	2/4/2/2	20	[-30, 60]

Outcomes list both successful / first only / second only / neither successful.

S8.2 Grid suite exclusions

Excluding Xin-She Yang 3 and Bohachevsky 2 preserves the main descriptive pattern. For the sampled Bohachevsky 2 lattice, $\cos(3\pi a_i) = 1$ at every coordinate, leaving an objective nearly redundant with sampled Sum of Squares after normalization. Without both functions, $B = 100$ rates are 79.5% for `RBFOpt`, 66.8% for `BAM`, 57.9% for `FMQA-SA`, 54.7% for the `BAM`-prefix control, 42.6% for `D-SDA`, and 17.9% for uniform. At $B = 10$, `BAM` and its prefix control each retain 47.9%, compared with `RBFOpt` 15.8%.

S8.3 Limited reconstructed FMQA diagnostics

Seven FM models were reconstructed locally from archived observations in three selected runs because the original fitted weights and QUBOs were not saved (SECQUOIA, 2026). All 3,328 enumerated binary states passed the recorded energy-conversion checks. Five selected SA proposals minimized their reconstructed models over eligible candidates; two `Dynamic` fallback iterations had no eligible stored SA sample. These selected checks do not verify the historical weights or isolate the contributions of learning, annealing, and fallback.